Informe de cobertura de testing

HuertoHogar · Evaluación Parcial 2

Equipo: Sebastian Monsalve y Lucas Parra. Asignatura: DSY1104. Ejecución verificada: 21 de octubre de 2026. Versión frontend: 2.0.0.

1. Resultado ejecutivo

Se ejecutaron 58 pruebas: 58 aprobadas y 0 fallidas. Se utilizó Jasmine para las aserciones y espías, Karma como ejecutor y Chrome Headless 148 como navegador de pruebas. La compilación de producción con Vite terminó correctamente. Los valores siguientes provienen de la ejecución de la suite actualizada.

Métrica	Cubierto / total	Porcentaje	Umbral mínimo
Sentencias	363 / 403	90,07%	90%
Ramas	336 / 386	87,04%	85%
Funciones	119 / 137	86,86%	85%
Líneas ejecutables	299 / 317	94,32%	90%

Los cuatro indicadores superan los umbrales definidos en coverageReporter.check de karma.conf.cjs. Una prueba fallida o una cobertura global inferior al umbral provoca una ejecución no exitosa.

2. Alcance del testing

Se verifica la lógica de cantidades, descuentos, total y validaciones; renderizado y manipulación del DOM; almacenamiento local y errores simulados; navegación, registro, contacto, protección visual de administración, ofertas y checkout. Se incluyen pruebas unitarias y algunas pruebas de integración de componentes con proveedor y rutas.

La instrumentación contempla los módulos de src cargados durante las pruebas. Se excluyen src/principal.jsx, src/productos.js, CSS, recursos multimedia, dependencias y configuración. La cobertura no representa un sistema de pagos real, un backend, ni todas las interacciones posibles.

3. Matriz de pruebas y requisitos

Grupo	Casos	Comportamientos comprobados	Requisitos
01 Carrito puro	5	Acumulación, inmutabilidad, cantidades, stock, total y CLP	RF05–RF06
02 ProductCard	3	Props, imagen, stock, mock y botón agotado	RF02, RF05
03 Catálogo	3	Productos, filtro, búsqueda, vacío y agregar	RF02–RF03, RF05
04 Detalle	2	Ruta, cantidad y producto inexistente	RF04–RF05
05 Carrito DOM	4	Vacío, subtotal, eliminar, vaciar y paso a checkout	RF06, RF14
06 Validaciones	3	Dominios, RUN, obligatorios, comuna y límites	RF08, RF10
07 Registro	4	Errores, cuenta válida, duplicados y guardado fallido	RF08
08 Sesión	3	Credenciales, cliente simulado y administrador	RF09
09 Contacto	3	Campos vacíos, guardado y error de persistencia	RF10
10 Estado y Layout	5	Menú, contador, sesión, avisos y persistencia	RF01, RF07, RF09
11 Administración	6	Acceso, CRUD, duplicados, stock y roles	RF12–RF13
12 Contenido	4	Inicio, video, Nosotros, blog y rutas inválidas	RF01, RF11
13 Aplicación completa	1	Proveedor, rutas y navegación entre formularios	RF01, RF08–RF10
14 Validaciones y ofertas	3	Correos inválidos, confirmación, dirección y descuentos	Validaciones y ofertas
15 Checkout y validaciones adicionales	9	Reglas detalladas, descuentos, renderizado y errores de checkout	Checkout y ofertas
TOTAL	58	58 casos aprobados	Frontend evaluado

4. Entorno y configuración

Las dependencias se definen en package.json y package-lock.json. Karma carga tests/tienda.spec.jsx; Webpack y Babel convierten JSX; babel-plugin-istanbul instrumenta el código; karma-coverage genera informes HTML, LCOV y JSON. React Testing Library renderiza componentes con MemoryRouter y StoreProvider cuando corresponde.

Karma utiliza Jasmine y ChromeHeadless. Puppeteer puede proporcionar un navegador de pruebas cuando CHROME_BIN no está definido. El archivo tests/preparacion.js prepara el entorno React y limpia DOM y localStorage entre casos para reducir interferencias.

5. Diseño de casos y uso de mocks

Las pruebas siguen la secuencia preparar, actuar y verificar (Arrange–Act–Assert): crear datos, renderizar, disparar eventos con fireEvent y comprobar resultados con expect. Incluyen escenarios correctos y errores como cantidades negativas o decimales, stock insuficiente, correos inválidos, RUN y cuentas duplicadas, direcciones incorrectas y fallos de almacenamiento.

Mock de callback: jasmine.createSpy permite comprobar que ProductCard comunique el producto esperado y que el botón agotado no invoque la acción.

Mock de repositorio: spyOn(storage, "read").and.callFake simula un usuario conocido para verificar el inicio de sesión sin depender de cuentas creadas en otras pruebas.

Mock de error: spyOn(storage, "write").and.returnValue(false) y la simulación de un error de setItem permiten verificar que la interfaz informe fallos de almacenamiento.

6. Resultados por módulo

La ejecución más reciente confirma los resultados globales de cobertura. Los porcentajes por archivo mostrados en el informe de la versión anterior (46 casos) ya no representan el código actual, porque se incorporaron Checkout.jsx, Ofertas.jsx y nuevas rutas de validación.

Indicador global actual	Valor comprobado
Líneas ejecutables	94,32% (299 / 317)
Ramas	87,04% (336 / 386)
Funciones	86,86% (119 / 137)
Sentencias	90,07% (363 / 403)

Para consultar cobertura individual de Checkout.jsx, Ofertas.jsx, Administracion.jsx y el resto, ejecutar npm.cmd test y abrir coverage/html/index.html. No se incluyen cifras por módulo no verificadas.

7. Análisis y límites de cobertura

La cobertura global de líneas fue del 94,32%, por lo que 18 de 317 líneas ejecutables no fueron recorridas por la suite. También quedan condiciones y funciones sin ejecutar en todas sus variantes. Esto es compatible con la aprobación de los cuatro umbrales configurados y no equivale a cobertura completa.

Las pruebas miden ejecución de código, pero no garantizan ausencia de defectos. No prueban pagos reales, envío efectivo de correos, autorización con servidor, sesiones compartidas entre dispositivos, todas las comunas de Chile, navegadores distintos de Chromium o una auditoría integral de accesibilidad.

8. Verificación visual y de navegación complementaria

En la revisión anterior se recorrieron ocho rutas (Inicio, catálogo, detalle FR001, carrito, registro, contacto, blog y Nosotros) a 390, 768 y 1440 píxeles: 24 comprobaciones. También se examinó el flujo catálogo–carrito. Estos resultados se conservan como antecedente histórico de esa versión y no se presentan como una nueva revisión visual completa de las funciones de checkout y ofertas.

Para cerrar la revisión visual de la versión actual, corresponde comprobar manualmente las pantallas nuevas de ofertas, checkout, resultado exitoso o fallido, mensajes rojos de validación y administrador en tamaños móvil y escritorio. Estas comprobaciones no forman parte de las 58 pruebas automáticas.

9. Reproducir y analizar

Desde la carpeta raíz del proyecto, instalar dependencias con npm.cmd install, compilar con npm.cmd run build y ejecutar pruebas mediante npm.cmd test. En la ejecución del 21 de octubre de 2026 se obtuvo TOTAL: 58 SUCCESS, sin fallos. Las cuatro métricas superaron sus umbrales.

El informe por archivo se genera en coverage/html/index.html; el resumen de máquina se encuentra en coverage/coverage-summary.json y el reporte LCOV en coverage/lcov.info. Estos archivos pueden regenerarse para revisar rutas y condiciones pendientes.

El proceso consistió en desarrollar componentes y casos de prueba, adaptar los tests al checkout y descuentos nuevos, ejecutar la suite, revisar los resultados y verificar la compilación. Las advertencias sobre dependencias de npm se revisan por separado y no modifican los resultados de prueba aquí indicados.